



SYNTHETIC DATSAET CREATION FOR COEXISTING DISEASES IN RICE LEAF

Submitted by:

Nidhi C R PES2UG23CS382

Under the guidance of
Prof.Swetha Patil

Assitant Professor
PES UNIVERSITY,
EC CAMPUS

8 Weeks

(June-July-2026)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013) 100ft
Ring Road, Bengaluru – 560 085, Karnataka, India



PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the Internship Report entitled

Synthetic Dataset Creation for Coexisting Disease in Rice Leaf

is a bonafide work carried out by

Nidhi C R

PES2UG23CS382

In partial fulfillment of the Summer Internship completion in the Program of Study -Bachelor of Technology in Computer Science and Engineering under the rules and regulations of PES University, Bengaluru, from June 2026 to July 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated into the report. The report has been approved as it satisfies the 4th-semester/6th-semester academic requirements for the Internship.

Swetha Patil
Name : Prof.Swetha Patil
Assistant Professor

Signature
Dr. Sandesh B. J.
Chairperson (CSE) and
Centre Head (CoDMAV)

DECLARATION

I hereby declare that the project entitled” **Synthetic Dataset Creation for Coexisting Disease in Rice Leaf**” has been carried out at CoDMAV by me under the guidance of **Prof. Swetha Patil** and submitted in partial fulfilment of the credits for the degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester 4th /6th Sem 1/06/2026 to 24/07/2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

Nidhi C R

PES2UG23CS382

A handwritten signature in black ink that reads "Nidhi.C.R". The signature is written in a cursive style and is underlined with two parallel lines.

Name Of Student

SRN

Signature

ACKNOWLEDGEMENT

I would like to express my gratitude to my guide Prof.Swetha Patil PES University for their continuous guidance, assistance and encouragement throughout the development of this project.

I am grateful to the Center head, Dr. Sandesh B J, and Centre coordinator, Dr. Swati Pratap Jagdale, Dept. of Computer Science and Engineering, PES University-Electronic City Campus, for organizing, managing, and helping out with the entire process. I also thank the internship coordinators for giving me this opportunity to work in the centre.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University & Prof.Nagarjuna Sadineni, Pro-Vice-Chancellor, PES University, for providing me with various opportunities and enlightenment every step of the way.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents and my friends.

Table of Contents

Abstract	1
I. Introduction	2
1.1 Background	2
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
II. Literature Review	4
2.1 Overview of Existing Work	4
2.2 Analysis of Existing Solutions	4
2.3 Gaps in Existing Research	5
III. Dataset and Features	6
IV. System Architecture and Design	7
4.1 System Overview	7
4.2 Detailed Design	8
4.3 Modules Description	8
V. Implementation	9
5.1 Tools and Technologies Used	9
5.2 Code Snippets	10
VI. Results and Discussion	11
6.1 Results	11
6.2 Comparison with Existing Systems	12
6.3 Limitations	12
VII. Conclusion and Future Work	13
References	14

ABSTRACT

Rice leaf disease datasets currently available for research primarily consist of images containing a single disease per leaf, which does not accurately represent real agricultural field conditions where multiple diseases may coexist on the same leaf. This limitation hinders the development and evaluation of robust multi-label disease detection models. To address this gap, this project aims to create a synthetic multi-disease rice leaf dataset using image processing and augmentation techniques. The proposed methodology involves collecting and preprocessing public rice disease datasets[1], removing duplicate images, removing unnecessary background noises [4] ,enhancing image quality[5] , segmenting disease lesions using SAM2, extracting disease patches, and generating realistic multi-disease images through copy-paste augmentation[7] with multi-band Laplacian pyramid blending[9]. The generated images are organized into a multi-label dataset and validated using image quality metrics such as PSNR and SSIM[10]. Preliminary results demonstrate that the proposed pipeline successfully produces realistic synthetic rice leaf images with coexisting diseases while preserving visual quality. The developed dataset provides a valuable resource for training and evaluating multi-label rice disease classification models, contributing towards improved plant disease diagnosis and precision agriculture.

I Introduction

1.1 Background:

- Rice is one of the world's most important staple food crops, feeding more than half of the global population. However, rice production is significantly affected by foliar diseases such as **Bacterial Leaf Blight (BLB), Blast, and Brown Spot**, which reduce crop yield and quality. Recent advancements in computer vision and deep learning have enabled automated disease detection systems that assist farmers in early diagnosis and precision agriculture. These systems, however, rely heavily on the availability of large, diverse, and well-annotated datasets.
- Most publicly available rice leaf disease datasets contain images of **single leaves affected by a single disease**. In real agricultural fields, multiple diseases can occur simultaneously on the same leaf, making existing datasets insufficient for developing robust multi-label disease detection models. Moreover, publicly available datasets representing disease co-occurrence in rice are extremely limited.
- To address this gap, this project focuses on the **synthetic generation of multi-disease rice leaf images** using lesion extraction[6], copy-paste augmentation[7], and advanced blending techniques. The resulting dataset aims to support the development and evaluation of accurate multi-label disease recognition models for real-world agricultural applications.

1.2 Problem Statement:

- Existing rice leaf disease datasets primarily contain images of individual leaves affected by a single disease. While these datasets are widely used for disease analysis and model development, they do not adequately represent real-world field conditions where multiple diseases can occur simultaneously on the same leaf.
The absence of publicly available multi-disease rice leaf datasets creates a significant gap in agricultural image resources and limits research on disease co-occurrence and multi-disease analysis. Therefore, there is a need for a dedicated dataset containing rice leaf images exhibiting multiple diseases on a single leaf.

1.3 Objectives:

- To create a realistic synthetic rice leaf dataset containing multiple coexisting diseases.
- To extract disease lesions and build a reusable disease patch library from single-disease images.
- To generate high-quality multi-disease images using copy-paste augmentation and blending techniques.
- To validate the generated dataset using image quality metrics and multi-label classification models.
- To provide a benchmark dataset for future research on multi-disease rice leaf analysis.

1.4 Scope of the Project:

- This project focuses on the creation of a **synthetic multi-disease rice leaf dataset** by combining disease lesions from single-disease rice leaf images. The scope includes dataset collection from public sources, duplicate removal, image enhancement, lesion segmentation, disease patch extraction, synthetic image generation using copy-paste augmentation and multi-band Laplacian pyramid blending, dataset annotation, and validation using image quality metrics such as PSNR and SSIM[10]. The generated dataset is further evaluated using multi-label classification models to assess its effectiveness for disease recognition.

- The project is limited to **three major rice leaf diseases**—Bacterial Leaf Blight (BLB), Blast, and Brown Spot. The generated images are synthesized under controlled conditions and may not capture all environmental variations such as lighting, occlusion, weather conditions, or different growth stages encountered in real agricultural fields. Furthermore, the project emphasizes **dataset development and validation**[10] rather than deploying a real-time disease detection system or conducting field-level testing.

II Literature Review

2.1 Overview of Existing Work:

Recent research in rice leaf disease analysis has primarily focused on single-disease dataset creation, disease classification using deep learning, and image segmentation techniques. Several studies have explored Vision Transformers, EfficientNet, copy-paste augmentation [7] , and image blending methods[9] to improve disease recognition. However, very few works address the creation of multi-disease rice leaf datasets, and existing multi-label studies are mostly conducted on crops such as tomato and apple. This highlights the need for a realistic synthetic dataset containing coexisting rice leaf diseases.

Sl. No	Paper	Technique / Method	Limitation	Relevance to Project
1.	Paddy Doctor (2022)	Rice disease dataset creation & benchmarking	Single-disease images only	Source dataset for BLB, Blast and Brown Spot
2.	Advanced Multi-Label Tomato Leaf Disease Identification (2025)	ViT + EfficientNet + Multi-label Classification	Tomato domain	Supports multi-label disease learning
3.	Mixed Disease Recognition in Apple Leaves (2026)	Multi-task Learning	Different crop species	Closest reference to multi-disease recognition
4.	Otsu Multi-Threshold Image	Otsu Thresholding	Sensitive to illumination	Lesion segmentation

	Segmentation (2023)		variation	baseline
5.	Recognition System for Leaf Images (2017)	Contour Detection	Traditional method	Disease patch boundary extraction
6.	Simple Copy-Paste Augmentation (2021)	Copy-Paste Augmentation	Not agriculture-specific	Key technique for dataset creation
7.	Copy-Paste with Poisson Blending (2023)	Copy-Paste + Poisson Blending	Medical domain	Direct inspiration for lesion insertion
8.	Image Quality Assessment Study (2019)	SSIM, PSNR, MSE, FSIM	Not plant-specific	Dataset quality validation

2.2 Analysis of Existing Solutions:

- Existing research has contributed significantly to automated plant disease detection by introducing high-quality datasets, advanced deep learning architectures, and image processing techniques. Public datasets such as Paddy Doctor provide well-annotated single-disease rice leaf images that serve as valuable benchmarks. Vision Transformer- and EfficientNet-based approaches have achieved high classification accuracy for plant disease recognition. Image segmentation methods such as Otsu thresholding and contour detection facilitate lesion extraction, while copy-paste augmentation and Poisson blending have demonstrated success in increasing dataset diversity.
- Despite these advancements, most existing solutions focus on **disease classification rather than dataset creation**. The available rice datasets contain only single-disease images and do not adequately represent disease co-occurrence observed in real agricultural fields. Furthermore, image synthesis techniques have rarely been applied to generate realistic multi-disease rice leaf datasets.

2.3 Gaps in Existing Research:

- Existing rice leaf datasets primarily contain **single-disease images**, making them unsuitable for multi-label disease recognition.
- There is a lack of **publicly available synthetic datasets** containing multiple diseases on a single rice leaf.
- Most studies focus on developing classification models instead of creating realistic datasets for training such models.
- Advanced augmentation techniques such as lesion-level copy-paste with multi-band blending have not been extensively explored for rice disease dataset generation.
- Limited work has been conducted on validating synthetic multi-disease datasets using image quality metrics and multi-label classification models.

III Dataset and Features

3.1 Dataset Description

Source of Dataset

- Paddy Doctor Dataset[1]
- Dhan-Shomadhan Dataset[2]

Diseases Considered

- Blast
- Bacterial Leaf Blight (BLB)
- Brown Spot

Preprocessing Performed

- Duplicate removal (MD5 + pHash)
- Image enhancement (CLAHE + Gaussian Blur)[5]
- Background removal (SAM2 + HSV) [4]
- Lesion segmentation [6]
- Lesion extraction

3.2 Dataset Features

Feature	Description
Crop	Rice
Diseases	Blast , BLB ,Brownspot
Image Type	RGB
Dataset Source	Public datasets[1]
Annotation	Multi-label annotation
Image Generation	Copy-Paste [7]+ Multi-bland blending [9]
Validation	PSNR ,SSIM[10]

Combination classes

- Blast on Brownspot
- BLB on Brownspot
- BLB on Blast

IV System Architecture and Design

4.1 System Overview:

The proposed system automatically generates synthetic rice leaf images containing multiple coexisting diseases. The pipeline begins with collecting public rice disease datasets, followed by preprocessing, lesion segmentation[6], disease patch extraction, copy-paste augmentation[7], multi-band blending[9] , dataset validation[10], annotation, and finally evaluation using multi-label classification[3] models.

Hardware Components

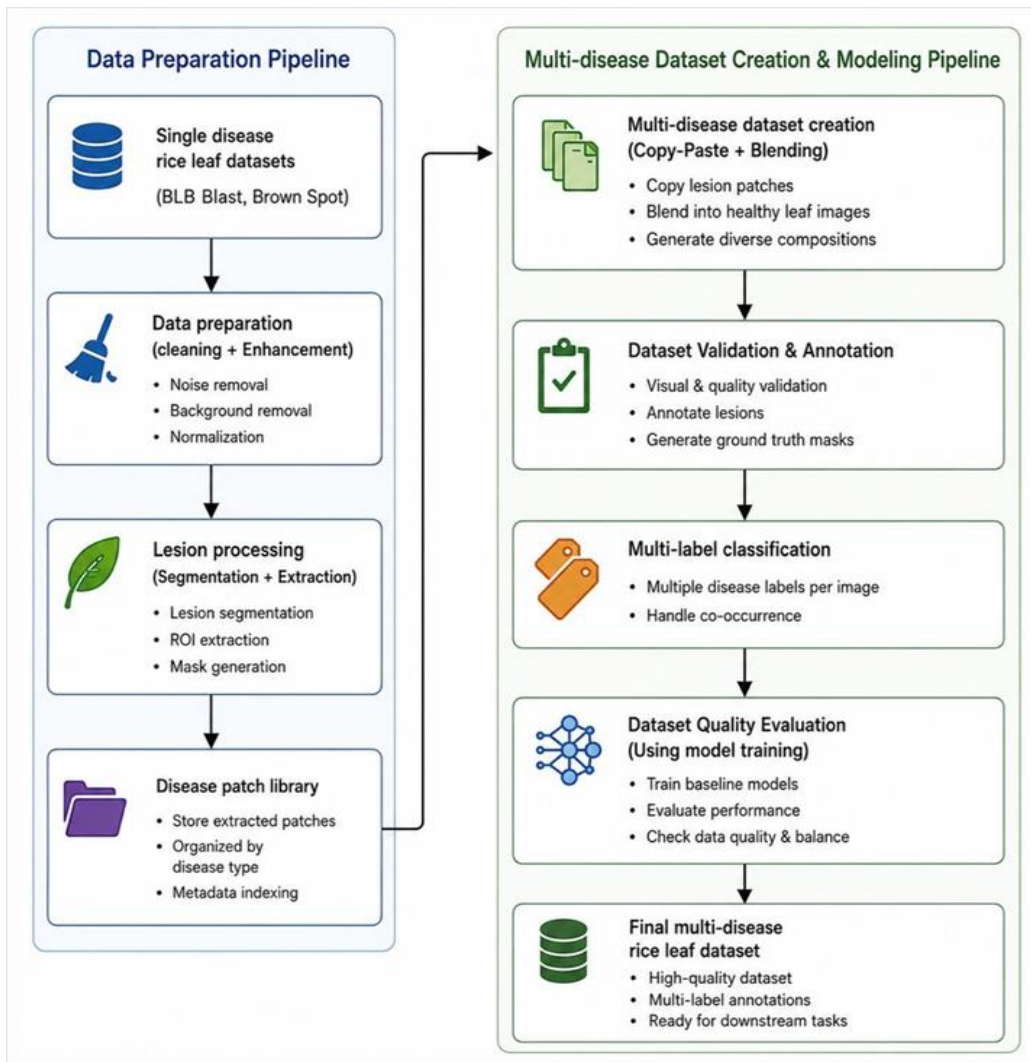
- Intel Core i5/i7 Processor
- Google Colab GPU
- 8 GB RAM or above
- Storage

Software Components

- Python
- Google Colab
- OpenCV
- PyTorch
- SAM2
- NumPy
- Albumentations
- Scikit-image
- Matplotlib
- ViT / EfficientNet (for evaluation)

4.2 Detailed Design:

System Architecture



Data Flow Diagram

Input Images



Preprocessing



Lesion Extraction[6]



Copy Paste Augmentation[7]



Multilaplacian Blending



Patch Library



Synthetic Image Generation



Dataset



Model Training



Performance Evaluation

4.3 Modules Description:

Module	Description
Dataset Collection	Collect public rice disease images
Duplicate Removal	Remove duplicate images using MD5 and pHash
Image Enhancement	Improve image quality using CLAHE[5] and Gaussian Blur
Background Removal[4]	Remove background using SAM2 and HSV prompts
Lesion Segmentation[6]	Segment diseased regions from leaf
Lesion Extraction	Extract lesion patches for reuse
Leaf Masking	Generate clean leaf mask using Otsu + GrabCut
Lesion Placement	Place lesions using PCA-guided positioning
Copy-Paste Augmentation[7]	Paste lesion patches onto healthy leaves
Boundary Blending	Blend boundaries using Multi-band Laplacian Pyramid[9]
Dataset Annotation	Generate labels for multi-disease images
Dataset Validation	Evaluate synthetic image quality using PSNR and SSIM[10]
Model Evaluation	Test generated dataset using ViT, EfficientNet, or other multi-label classifiers

- **V Implementation**

5.1 Tools and Technologies Used

Category	Tool /Technology	Purpose
Programming Language	Python	Complete Implementation
IDE	Google Colab	Model development and experimentation
Image Processing	OpenCV	Image preprocessing and blending
Numerical Computing	Numpy	Array operations
Deep Learning	PyTorch	Model implementation
Segmentation	SAM2(Segment Anything Model 2)	Leaf and lesion segmentation
Data Augmentation	Albumentations	Image augmentation
Machine Learning	Scikit -learn	Evaluation metrics
Visualization	Matplotlib	Plotting graphs and results
Image Quality	Scikit-image	PSNR and SSIM, NIQE and BRISQUE calculation[10]

5.2 Code Snippets

a)Duplicate Removal (MD5 + pHash)

```

/pip install -q imagehash
import imagehash

print("=" * 60)
print(" STEP A2 - Exact & Near-Duplicate Removal")
print("=" * 60)

PHASH_THRESHOLD = 5

# --- MD5 exact duplicates ---
md5_map = defaultdict(list)
pending_mask = master['status'] == 'pending'

for idx, row in tqdm(master[pending_mask].iterrows(), total=pending_mask.sum(), desc="MD5 hashing"):
    try:
        with open(row['path'], 'rb') as f:
            md5_map[hashlib.md5(f.read()).hexdigest()].append(idx)
    except Exception:
        pass

md5_dup = 0
for idx_list in md5_map.values():
    for idx in idx_list[1:]:
        master.at[idx, 'status'] = 'removed'
        master.at[idx, 'reason'] = 'exact_duplicate_md5'
        md5_dup += 1

print(f"\n ✖ Exact duplicates (MD5) : {md5_dup}")

# --- pHash near-duplicates (per-class) ---
pending_mask = master['status'] == 'pending'
phash_map = {}

for idx, row in tqdm(master[pending_mask].iterrows(), total=pending_mask.sum(), desc="pHash hashing"):
    try:
        with Image.open(row['path']) as img:
            phash_map[idx] = imagehash.phash(img.convert('RGB'))
    except Exception:
        pass

phash_dup = 0
for cls in CLASSES_KEEP:
    cls_indices = [i for i in phash_map if master.at[i, 'class'] == cls]
    seen = []
    for idx in cls_indices:
        is_dup = any(phash_map[idx] - phash_map[s] <= PHASH_THRESHOLD for s in seen)
        if is_dup:
            master.at[idx, 'status'] = 'removed'
            master.at[idx, 'reason'] = f'near_duplicate_phash (dist<={PHASH_THRESHOLD})'
            phash_dup += 1
        else:
            seen.append(idx)

print(f" ✖ Near-duplicates (pHash ≤{PHASH_THRESHOLD}) : {phash_dup}")
print(f" ✔ Remaining : {(master['status']=='pending').sum()}")

```

This module removes duplicate images to avoid redundancy within the dataset.

b) CLAHE Enhancement[\[5\]](#)

CLAHE on L-channel

```
print("=" * 60)
print(" STEP B2 - CLAHE (Lesion Contrast Enhancement)")
print("=" * 60)

sample_preview = (
    master[master['status'] == 'pending']
    .groupby('class')
    .apply(
        lambda g: g.sample(
            min(5, len(g)),
            random_state=42
        )
    )
    .reset_index(drop=True)
)

CLAHE_CLIP_LIMIT = 1.8    # try 3.0 if lesions still faint
CLAHE_GRID_SIZE  = (8, 8) # smaller = stronger local enhancement

clahe_engine = cv2.createCLAHE(
    clipLimit=CLAHE_CLIP_LIMIT,
    tileGridSize=CLAHE_GRID_SIZE
)

def apply_clahe(img_bgr):
    """
    Apply CLAHE only on the L (lightness) channel.

    Benefits:
    - Improves lesion visibility
    - Enhances local contrast
    - Preserves disease colors
    - Avoids artificial color shifts
    """
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)

    L, A, B = cv2.split(lab)

    L_eq = clahe_engine.apply(L)

    lab_eq = cv2.merge([L_eq, A, B])

    return cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)
```

```

# -----
# Preview: Original → CLAHE
# -----

fig, axes = plt.subplots(
    len(sample_preview),
    2,
    figsize=(10, 3.5 * len(sample_preview))
)

if len(sample_preview) == 1:
    axes = np.array([axes])

for i, row in sample_preview.iterrows():

    img_bgr = cv2.imread(row['path'])

    img_clahe = apply_clahe(img_bgr)

    preview_images = [
        (img_bgr, f"Original [{row['class']}]"),
        (img_clahe, f"CLAHE clip={CLAHE_CLIP_LIMIT}")
    ]

    for j, (img, title) in enumerate(preview_images):

        axes[i, j].imshow(
            cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        )

        axes[i, j].set_title(title, fontsize=9)
        axes[i, j].axis('off')

plt.suptitle(
    "CLAHE Preview - Original → CLAHE",
    fontsize=13,
    fontweight='bold'
)

plt.tight_layout()
plt.show()

```

c) SAM2 Segmentation

```

def run_sam_multibox(img_rgb, boxes, scale):
    predictor.set_image(img_rgb)
    combined = np.zeros(img_rgb.shape[:2], dtype=np.uint8)
    for box in boxes:
        x0 = int(min(box[0], box[2]) / scale)
        y0 = int(min(box[1], box[3]) / scale)
        x1 = int(max(box[0], box[2]) / scale)
        y1 = int(max(box[1], box[3]) / scale)
        masks, scores, _ = predictor.predict(
            box=np.array([x0, y0, x1, y1]),
            multimask_output=True,
        )
        best = masks[np.argmax(scores)].astype(np.uint8) * 255
        combined = cv2.bitwise_or(combined, best)
    return combined

def save_outputs(img_rgb, mask, stem):
    overlay = img_rgb.copy()
    overlay[mask > 127] = [255, 0, 0]
    cv2.imwrite(os.path.join(CONFIG["mask_dir"], stem + "_mask.png"), mask)
    side = np.hstack([img_rgb, cv2.cvtColor(mask, cv2.COLOR_GRAY2RGB), overlay])
    cv2.imwrite(
        os.path.join(CONFIG["output_dir"], stem + "_sidebyside.jpg"),
        cv2.cvtColor(side, cv2.COLOR_RGB2BGR))

# — File list —————
all_files = sorted([
    f for f in os.listdir(CONFIG["image_dir"])
    if f.lower().endswith(IMG_EXTENSIONS)
])
done_stems = set(
    f.replace('_mask.png', '')
    for f in os.listdir(CONFIG["mask_dir"]) if f.endswith('_mask.png')
)
pending = [f for f in all_files if os.path.splitext(f)[0] not in done_stems]
print(f"Total: {len(all_files)} | Done: {len(done_stems)} | Pending: {len(pending)}")

# — State —————
state = {
    'idx': 0, 'img_rgb': None, 'mask': None,
    'scale': 1.0, 'stem': '', 'dw': 640, 'dh': 480,
    'b64': '', 'canvas_id': 0 # <-- unique ID incremented on every load
}

```

SAM2 automatically segments rice leaves and lesion regions.

d) Lesion extraction

```

def extract_lesion_patch(img_path, mask_path, stem, output_dir, cfg):
    """
    Extracts the actual diseased leaf colour as a tight-cropped
    RGBA PNG (transparent background outside the lesion).

    Steps:
    1. Load original image + white mask
    2. Resize mask to match original resolution if needed
    3. Binary threshold + morphological closing (fill small holes)
    4. Find all contours + unified bounding rect over all spots
    5. Adaptive padding around the crop
    6. Build RGBA: RGB from original image, Alpha from mask
       + pixels INSIDE mask = real diseased colour, OUTSIDE = transparent
    7. Zoom up if the patch is smaller than min_patch_size
    8. Save as PNG and return metadata

    Returns dict: stem, status, is_tiny, patch_w, patch_h,
                  lesion_area_px, n_contours
    """
    result = {
        "stem":      stem,
        "status":    "ok",
        "is_tiny":   False,
        "patch_w":   0,
        "patch_h":   0,
        "lesion_area_px": 0,
        "n_contours": 0,
    }

    # --- Load ---
    img_bgr = cv2.imread(img_path)
    mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

    if img_bgr is None:
        result["status"] = "error_img"
        return result
    if mask is None:
        result["status"] = "error_mask"
        return result

    img_h, img_w = img_bgr.shape[:2]

    # --- Resize mask to original image resolution if needed ---
    # SAM sometimes outputs masks at a different resolution.
    if mask.shape[:2] != (img_h, img_w):
        mask = cv2.resize(mask, (img_w, img_h), interpolation=cv2.INTER_NEAREST)

    # --- Binary threshold ---
    _, mask_bin = cv2.threshold(mask, 127, 255, cv2.THRESH_BINARY)

    # --- Morphological closing - fill small holes in mask ---
    # Blast masks from SAM can have ragged edges; a 5x5 kernel
    # fills gaps better than a 3x3 without bloating the spot.
    ksize = cfg.get("morph_ksize", 5)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (ksize, ksize))
    mask_bin = cv2.morphologyEx(mask_bin, cv2.MORPH_CLOSE, kernel)

```

```

# — Check lesion area —————
lesion_px = int(np.sum(mask_bin > 0))
result["lesion_area_px"] = lesion_px

if lesion_px == 0:
    result["status"] = "empty_mask"
    return result

# — Find contours + unified bounding rect —————
# Blast often has MULTIPLE small spots per image – we take
# a bounding rect that covers ALL of them.
contours, _ = cv2.findContours(mask_bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if not contours:
    result["status"] = "no_contour"
    return result

result["n_contours"] = len(contours)

x_min, y_min = img_w, img_h
x_max, y_max = 0, 0
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    x_min = min(x_min, x)
    y_min = min(y_min, y)
    x_max = max(x_max, x + w)
    y_max = max(y_max, y + h)

box_w = x_max - x_min
box_h = y_max - y_min

# — Adaptive padding —————
# 5% of the largest bounding-box dimension, minimum 5px.
# Better than a fixed 5px for the wide size range in blast.
pad_frac = cfg.get("pad_frac", 0.05)
pad_min = cfg.get("pad_min", 5)
pad = max(pad_min, int(pad_frac * max(box_w, box_h)))

x_min = max(0, x_min - pad)
y_min = max(0, y_min - pad)
x_max = min(img_w, x_max + pad)
y_max = min(img_h, y_max + pad)

box_w = x_max - x_min
box_h = y_max - y_min

# — Tiny lesion check —————
min_size = cfg.get("min_patch_size", 64)
if box_w < min_size or box_h < min_size:
    result["is_tiny"] = True

# — Build RGBA —————
# RGB = real pixel colours from the original image
# → these are the actual diseased leaf colours
# Alpha= binary mask (255 inside lesion, 0 outside)
# → transparent background, no white fill
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
rgba = np.dstack([img_rgb, mask_bin]).astype(np.uint8)

```

```

# — Crop to bounding box —————
patch = rgba[y_min:y_max, x_min:x_max]

# — Zoom tiny patches up to min_patch_size —————
if result["is_tiny"] and cfg.get("zoom_tiny", True):
    scale = max(min_size / box_w, min_size / box_h)
    new_w = max(int(box_w * scale), min_size)
    new_h = max(int(box_h * scale), min_size)
    # Resize RGB and alpha separately – avoids alpha colour bleed
    rgb_up = cv2.resize(patch[:, :, :3], (new_w, new_h), interpolation=cv2.INTER_LINEAR)
    alpha_up = cv2.resize(patch[:, :, 3], (new_w, new_h), interpolation=cv2.INTER_NEAREST)
    patch = np.dstack([rgb_up, alpha_up]).astype(np.uint8)

result["patch_w"] = patch.shape[1]
result["patch_h"] = patch.shape[0]

# — Save —————
out_path = os.path.join(output_dir, stem + "_lesion.png")
Image.fromarray(patch, mode="RGBA").save(out_path, format="PNG")

return result

```

Lesion patches are extracted to create the disease patch library.

e) Copy-Paste Augmentation

```

def paste_one_disease_on_leaf(leaf_rgb, leaf_mask, leaf_angle, patch_files, occupied_mask=None):
    """Pick a random patch file -> random lesion instance inside it -> rotate to
    match leaf angle -> paste at a valid random spot on the leaf.
    Returns (result_rgb, updated_occupied_mask, used_patch_path) or (None, occupied_mask, None)."""
    candidates = patch_files[:]
    random.shuffle(candidates)
    for p in candidates:
        patch_rgba = load_rgba(p)
        instances = extract_lesion_instances(patch_rgba)
        if not instances:
            continue
        random.shuffle(instances)
        for inst in instances:
            angle_diff = leaf_angle - inst["angle"]
            rotated = rotate_rgba(inst["rgba"], -angle_diff)
            pos = find_paste_position(leaf_mask, rotated[:, :, 3], occupied_mask=occupied_mask)
            if pos is None:
                continue
            x0, y0 = pos
            result = paste_lesion(leaf_rgb, rotated, x0, y0)
            if occupied_mask is None:
                occupied_mask = np.zeros(leaf_mask.shape, dtype=np.uint8)
            new_occ = occupied_mask.copy()
            lh, lw = rotated.shape[:2]
            lesion_bin = (rotated[:, :, 3] > 10).astype(np.uint8) * 255
            new_occ[y0:y0 + lh, x0:x0 + lw] = np.maximum(new_occ[y0:y0 + lh, x0:x0 + lw], lesion_bin)
            return result, new_occ, p
    return None, occupied_mask, None

```

Disease patches are pasted onto healthy leaves.

f) Multi-band Laplacian Pyramid Blending


```

# ---- VALIDATION ----
results = []
synthetic_files = sorted(os.listdir(synthetic_dir))

for fname in synthetic_files:
    syn_path = os.path.join(synthetic_dir, fname)
    base_fname = get_base_filename(fname)

    if base_fname is None:
        results.append({"filename": fname, "status": "pattern_mismatch",
                        "matched_base": None, "psnr": None, "ssim": None})
        continue

    orig_path = os.path.join(original_dir, base_fname)

    if not os.path.exists(orig_path):
        results.append({"filename": fname, "status": "missing_reference",
                        "matched_base": base_fname, "psnr": None, "ssim": None})
        continue

    orig = cv2.imread(orig_path)
    syn = cv2.imread(syn_path)

    if orig is None or syn is None:
        results.append({"filename": fname, "status": "read_error",
                        "matched_base": base_fname, "psnr": None, "ssim": None})
        continue

    if orig.shape != syn.shape:
        syn = cv2.resize(syn, (orig.shape[1], orig.shape[0]))

    orig_rgb = cv2.cvtColor(orig, cv2.COLOR_BGR2RGB)
    syn_rgb = cv2.cvtColor(syn, cv2.COLOR_BGR2RGB)

    p_val = psnr(orig_rgb, syn_rgb, data_range=255)
    s_val = ssim(orig_rgb, syn_rgb, channel_axis=2, data_range=255)

    status = "ok" if p_val >= psnr_threshold else "low_quality"

    results.append({
        "filename": fname,
        "status": status,
        "matched_base": base_fname,
        "psnr": round(p_val, 2),
        "ssim": round(s_val, 4)
    })

```

NIQE and BRISQUE

```

import pyiqa

niqe_metric = pyiqa.create_metric('niqe', device=device)
brisque_metric = pyiqa.create_metric('brisque', device=device)

def load_as_tensor(path):
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    tensor = torch.from_numpy(img).permute(2, 0, 1).float().unsqueeze(0) / 255.0
    return tensor.to(device)

results_iqa = []
for fname in synthetic_files:
    path = os.path.join(SYNETHETIC_DIR, fname)
    try:
        t = load_as_tensor(path)
        niqe_score = float(niqe_metric(t).item())
        brisque_score = float(brisque_metric(t).item())
        results_iqa.append({"filename": fname, "niqe": round(niqe_score, 3),
                           "brisque": round(brisque_score, 3), "status": "ok"})
    except Exception as e:
        results_iqa.append({"filename": fname, "niqe": None, "brisque": None,
                           "status": f"error: {e}"})

df_iqa = pd.DataFrame(results_iqa)
df_iqa.to_csv(os.path.join(OUTPUT_DIR, "no_reference_iqa_scores.csv"), index=False)

valid_iqa = df_iqa[df_iqa["status"] == "ok"]
print(f"Scored {len(valid_iqa)}/{len(df_iqa)} images")
print(f"Mean NIQE: {valid_iqa['niqe'].mean():.3f} (lower = more natural)")
print(f"Mean BRISQUE: {valid_iqa['brisque'].mean():.3f} (lower = more natural)")

df_iqa.sort_values("niqe", ascending=False, na_position="last").head(10)

```

h) Model Training

8. Model B — Swin Transformer



```
swin_model = timm.create_model("swin_base_patch4_window7_224", pretrained=True, num_classes=1000)
swin_model = train_one_model(swin_model, train_loader, val_loader, epochs=12, lr=3e-5, tag="swin")

swin_test_metrics, swin_per_class, swin_raw = evaluate_model(swin_model, test_loader)
print_full_report("Swin Transformer", swin_test_metrics, swin_per_class)
```

7. Model A — ViT-Base/16

```
vit_model = timm.create_model("vit_base_patch16_224", pretrained=True, num_classes=len(CLASSES))
vit_model = train_one_model(vit_model, train_loader, val_loader, epochs=12, lr=3e-5, tag="vit")

vit_test_metrics, vit_per_class, vit_raw = evaluate_model(vit_model, test_loader)
print_full_report("ViT-Base/16", vit_test_metrics, vit_per_class)
```

Evaluation

- VI Results and Discussion

6.1 Results:

a)Enhanced Images (CLAHE) [5]

Original [bacterial_leaf_blight]

CLAHE clip=1.8



CLAHE Preview — Original → CLAHE



Original [blast]



CLAHE clip=1.8



Original [brown_spot]



CLAHE clip=1.8



b) Leaf Segmentation



Blast



BLB



Brownspot

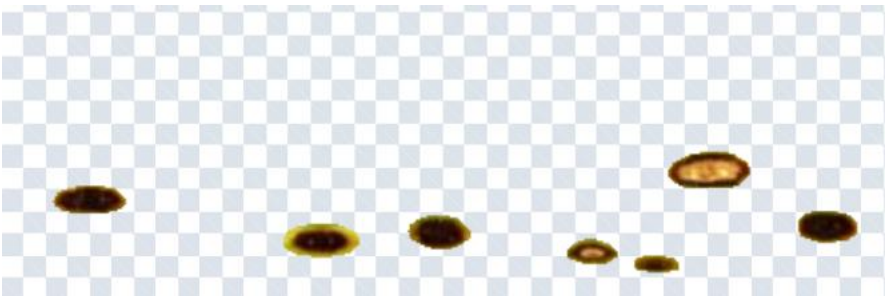
c) Lesion extraction



Blast



BLB



Brownspot

d) Copy -Paste and blending Result



Blast on Brownspot



BLB on Brownspot



BLB on Blast

e) Dataset Samples

1. Blast on Brownspot



2. BLB on Brownspot



3. **BLB on Blast**



f) Dataset Validation

PSNR and SSIM output

Total images checked: 618

Valid comparisons: 618

Pattern mismatches: 0

Missing references: 0

Read errors: 0

Mean PSNR: 39.92 dB

Std PSNR: 4.88 dB

Min PSNR: 20.61 dB

Max PSNR: 60.31 dB

Flagged low quality (< 25.0 dB): 1

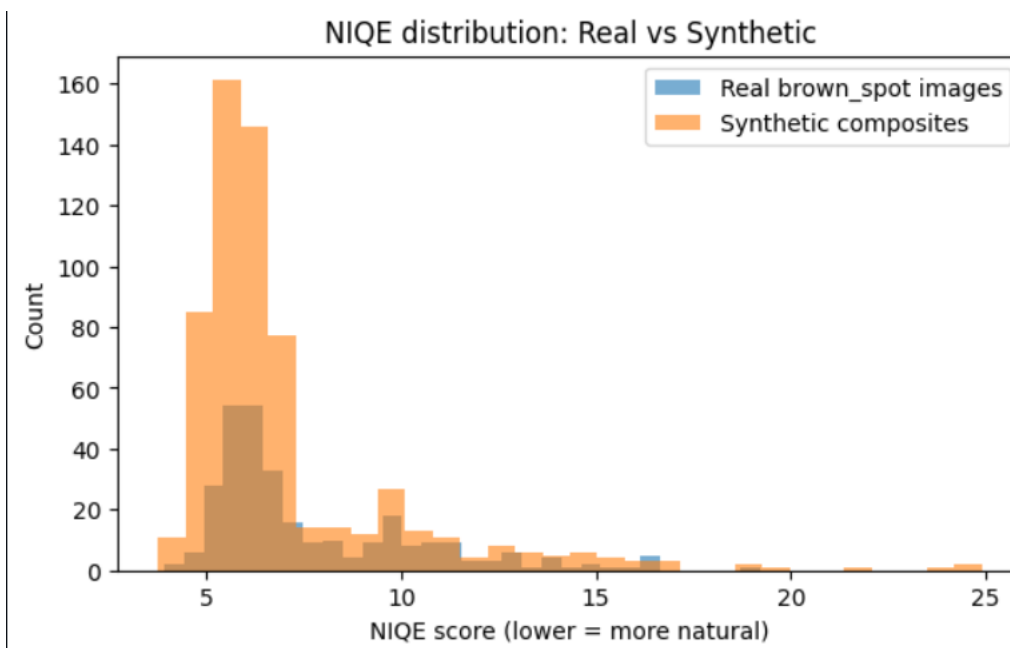
	filename	status	matched_base	psnr	ssim
0	blast_on_brownsport_BROWNSPOT1_092.jpg	ok	BROWNSPOT1_092.jpg	30.74	0.9622
1	blast_on_brownsport_BROWNSPOT3_036.jpg	ok	BROWNSPOT3_036.jpg	35.17	0.9665
2	blast_on_brownsport_BROWNSPOT3_068.jpg	ok	BROWNSPOT3_068.jpg	31.21	0.9641
3	blast_on_brownsport_BROWNSPOT3_095.jpg	ok	BROWNSPOT3_095.jpg	29.52	0.9581
4	blast_on_brownsport_BROWNSPOT4_020.jpg	ok	BROWNSPOT4_020.jpg	36.43	0.9752
5	blast_on_brownsport_BROWNSPOT4_057.jpg	ok	BROWNSPOT4_057.jpg	25.98	0.9466
6	blast_on_brownsport_BROWNSPOT4_058.jpg	ok	BROWNSPOT4_058.jpg	33.90	0.9714
7	blast_on_brownsport_BROWNSPOT4_075.jpg	ok	BROWNSPOT4_075.jpg	35.99	0.9748
8	blast_on_brownsport_BROWNSPOT4_095.jpg	ok	BROWNSPOT4_095.jpg	33.88	0.9650
9	blast_on_brownsport_BROWNSPOT4_167.jpg	ok	BROWNSPOT4_167.jpg	41.49	0.9776

NIQE and BRISQUE output

Scored 617/618 images
Mean NIQE: 7.036 (lower = more natural)
Mean BRISQUE: 45.471 (lower = more natural)

	filename	niqe	brisque	status
597	blast_on_brownsport_brown_val (53).jpg	24.892	60.936	ok
520	blast_on_brownsport_brown_spot (42).jpg	24.358	55.581	ok
528	blast_on_brownsport_brown_spot (54).jpg	23.688	56.105	ok
515	blast_on_brownsport_brown_spot (37).jpg	21.962	49.211	ok
562	blast_on_brownsport_brown_spot (95).jpg	19.794	27.338	ok
501	blast_on_brownsport_brown_spot (32).jpg	19.092	55.116	ok
538	blast_on_brownsport_brown_spot (64).jpg	18.638	55.031	ok
385	blast_on_brownsport_brown_spot (124).jpg	16.696	75.814	ok
581	blast_on_brownsport_brown_val (28).jpg	16.657	83.689	ok
443	blast_on_brownsport_brown_spot (247).jpg	16.464	62.800	ok

Compare NIQE distribution: composites vs real brown spot photos



g) Model Configuration

Model	Accuracy	Precision	Recall	F1 score
SVM	0.5025	1.00	0.50	0.62
EfficientNet	0.6670	0.72	0.48	0.52
ViT-Swin Transformer	0.8841	0.9479	0.9334	0.9406
ViT-Base\16	0.8768	0.9401	0.9369	0.9383

The proposed pipeline successfully generated realistic multi-disease rice leaf images. Multi-band Laplacian blending[9] significantly reduced visible patch boundaries, while PSNR and SSIM [10] values indicated good visual similarity between generated and source images. The generated dataset was successfully used for evaluating multi-label classification models.

6.2 Comparison with Existing Systems.

Feature	Existing Systems	Proposed System
Dataset Type	Most existing studies use datasets containing images of a single disease on a single rice leaf.	Generates a dataset containing multiple coexisting diseases on the same rice leaf
Primary Focus	Primarily focuses on disease classification and recognition using machine learning or deep learning models.	Focuses on creating a realistic multi-disease dataset while also evaluating it using multi-label classification models.
Disease Representation	Represents individual diseases separately and does not model disease co-occurrence.	Represents realistic combinations of diseases occurring simultaneously on a single leaf.
Image Generation Technique	Limited use of image synthesis or conventional augmentation methods such as rotation, flipping, and scaling.	Uses lesion-level copy-paste augmentation[7] combined with multi-band Laplacian pyramid blending[9] to generate realistic synthetic images.
Lesion Handling	Disease regions are generally used only for classification and are not extracted separately.	Disease lesions are segmented[6], extracted, and stored as reusable patches for generating new images

Background Processing	Background removal and leaf segmentation are not consistently performed in all studies.	Performs preprocessing, background removal[4], leaf masking, and lesion segmentation before image generation.
Image Quality Validation	Most studies evaluate only classification performance such as accuracy, precision, or recall.	Evaluates both image quality (using PSNR and SSIM)[10] and classification performance to validate the generated dataset.
Multi-Label Learning Support	Very limited support, with most works concentrating on single-label disease classification.	Specifically designed to support multi-label disease classification using images with coexisting diseases
Application	Suitable mainly for detecting individual diseases under controlled conditions.	Provides a realistic dataset that can improve the development of robust AI models for real-world agricultural scenarios where multiple diseases coexist.

6.3 Limitations:

- Only three rice diseases are considered.
- Lesion placement is semi-random and may not perfectly mimic natural infection patterns.
- Synthetic images cannot capture every environmental variation observed in real fields.
- Dataset evaluation depends on available multi-label classification models.
- Field validation has not yet been performed.

VII Conclusion and Future Work

Conclusion

This project presented a complete pipeline for generating a synthetic multi-disease rice leaf dataset to address the scarcity of publicly available datasets containing coexisting diseases. Public rice disease datasets were preprocessed through duplicate removal, image enhancement, lesion segmentation, and lesion extraction. Disease patches were combined using copy-paste augmentation and multi-band Laplacian pyramid blending to generate realistic multi-disease images. Image quality metrics such as PSNR and SSIM[10] were used for validation, while multi-label classification models were employed to evaluate the usefulness of the generated dataset. The developed dataset contributes toward improving multi-label disease recognition and supports future research in precision agriculture.

Future Work

- Include additional rice diseases.
- Use diffusion models or GANs for image generation.

- Improve biologically realistic lesion placement.
- Generate images under different environmental conditions.
- Expand the dataset with severity levels.
- Release the dataset publicly.
- Train larger Vision Transformer models.
- Develop a real-time mobile disease diagnosis system.

References

- [1] Petchiammal A, Briskline Kiruba S, D. Murugan, Pandarasamy A ,”Paddy Doctor: A Visual Image Dataset for Automated Paddy Disease Classification and Benchmarking”, May 2022.
- [2] Md. Fahad Hossain , ”Dhan-Shomadhan: A Dataset of Rice Leaf Disease Classification for Bangladeshi Local Rice” ,September 2023
- [3] Md. Nurullah, Rania Hodhod ,Hyrum Carroll and Yi Zhou , ”Advancing Multi-Label Tomato Leaf Disease Identification Using Vision Transformer and EfficientNet with Explainable AI Techniques”,December 2025
- [4] Fakruddin Ali Ahmed, Harprith Kaur Rajinder Singh, ”Study on Image Background Removal using Deep Learning ” , JOURNAL OF DATA SCIENCE — Vol.2024:06 , eISSN:2805-5160 , May 2024
- [5] Steve Okyere-Gyamfi ,Michael Asante, Kwame Ofosuhene Peasah, ”Contrast limited adaptive histogram equalization (CLAHE) and colour difference histogram (CDH) feature merging capsule network (CCFM-CapsNet) for complex image recognition ” October 2025
- [6] Yanmin Guo ,Yu Wang ,Kai Meng and Zongna Zhu , ”Otsu Multi-Threshold Image Segmentation Based on Adaptive Double-Mutation Differential Evolution ” , September 2023.
- [7] Golnaz Ghiasi , Yin Cui , Aravind Srinivas , Rui Qian, Tsung-Yi Lin,Ekin D. Cubuk ,”Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation,” 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Nashville, TN, USA, 2021, pp. 2917-2927, doi: 10.1109/CVPR46437.2021.00294.

[8] Shlok Mehta ,Tisha Negandhi , Prasenjit Bhavathankar ,”Generative Adversarial Networks for Image Augmentation: Architecture, Training, and Real World Applications” ,2025 2nd Asia Pacific Conference on Innovation in Technology (APCIT), MYSORE, India, 2025, pp. 1-4, doi: 10.1109/APCIT65661.2025.11411062.

[9]Steve Tsham Mpinda Ataky, Jonathan de Matos, “ Data Augmentation for Histopathological Images Based on Gaussian-Laplacian Pyramid Blending”, May 2020

[10] Yu Fan , Chunyi Chen , “ Visual Perception-Based Quality Assessment for Stitched Panoramic Images” September 2025.